

AI 大模型应用开发 — 学习笔记

学习方式：跟着教程一课一课实操，从零搭建可运行的 AI 项目 学习目标：AI 大模型应用开发工程师

第1课：Python 基础（看懂 AI 项目代码）

必须掌握的核心概念

概念	用途	在 AI 项目里的样子
变量	存数据	<code>text = "你好"、count = 42</code>
列表（List）	一堆有序数据	<code>["文本1", "文本2", "文本3"]</code>
字典（Dict）	带标签的数据	<code>{"role": "user", "content": "你好"}</code>
函数（def）	打包可复用的逻辑	<code>def call_model(prompt): ...</code>
JSON	AI 项目最通用的数据格式	模型输入输出、API 请求响应全是 JSON
import	引入别人写好的功能	<code>import json、import requests</code>

关键理解

AI 项目里的数据结构 = 列表套字典、字典套列表。看懂这两种嵌套，就能看懂大多数 AI 代码。

```
# 大模型聊天数据格式（列表套字典）
messages = [
    {"role": "system", "content": "你是一个助手"},
    {"role": "user", "content": "你好"}
]
```

第2课：FastAPI 后端接口

接口的本质

别人访问你的网址，你收到请求、处理一下、把结果还回去。

核心代码模板

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class 请求格式(BaseModel):
    参数名: str

@app.post("/接口地址")
def 函数名(req: 请求格式):
    # 处理逻辑
    return {"key": "value"}
```

关键理解

元素	作用
<code>@app.get("/")</code>	定义 GET 接口
<code>@app.post("/ask")</code>	定义 POST 接口（收数据用）
<code>return {}</code>	返回 JSON 给调用方
<code>req.参数名</code>	拿到用户传过来的参数

实用工具

- Swagger UI: <http://127.0.0.1:8000/docs> — 自动生成的可视化 API 调试页面
- uvicorn: 让 FastAPI 跑起来的服务器

第3课：提示词工程

5 个核心技巧

技巧	说明	示例
① 定角色	告诉模型"你是谁"	"你是一名资深 Python 工程师"
② 拆任务	复杂的活分步骤说	把一个大任务拆成 4 步
③ 固定输出格式	定死返回的 JSON 结构	方便程序解析，不做正则匹配
④ 少样本（Few-Shot）	给 1-2 个例子	模型通过例子理解你要的风格
⑤ 思维链（CoT）	让模型先推理再给答案	"第一步：... 第二步：..."

建议每次都用这个模板

角色 [你希望模型扮演什么角色] ## 背景 [这个任务在什么场景下使用] ## 任务 [具体要做什么，拆成步骤] ## 要求 [格式要求、约束条件] ## 输出格式 [定死结构，方便程序解析]
--

第4课：RAG（检索增强生成）

为什么需要 RAG？

问题	纯大模型	RAG
知识过时	训练完就"冻住"了	实时查最新资料回答

问题	纯大模型	RAG
胡编乱造（幻觉）	不知道也编	查了资料再回答

RAG = 开卷考试。先翻书，再答题。

完整 5 步流程

- | | |
|------------------------------------|--------|
| ① 文档提前转成向量（一次性建库） | ← 预处理 |
| ② 用户问题转成向量 | ← 每次提问 |
| ③ 向量数据库搜最接近的文档（算相似度） | ← 检索 |
| ④ 找到的文档 + 问题拼成 <code>prompt</code> | ← 拼接 |
| ⑤ 发给大模型回答（基于资料） | ← 生成 |

通俗理解

"工资什么时候发" → 转成向量 → 向量库找到 "薪资每月15日发放"（相似度 95%） → 拼在一起发给模型 → 模型"看了资料再回答"

关键：搜意思，不是搜关键词。 "工资" ≠ "薪资"，但向量认为它们意思相近。

第5课：向量搜索 — 搜意思而非搜关键词

什么是向量？

每个词/句子 → 转成一串数字（几百维） → 这串数字就是"意思的身份证"

"年假怎么休" → [0.12, 0.56, -0.33, 0.78, ...]
"我有几天假期" → [0.11, 0.55, -0.31, 0.76, ...] ← 数字接近，意思相近

技术组件

组件	作用
Embedding 模型	把文字转成向量（如 BGE、text-embedding-3）

组件	作用
向量数据库	存向量 + 搜最近邻（Chroma、FAISS、Milvus）
余弦相似度	两个向量之间"角度"越小越相关

第6课：调大模型 API

调 DeepSeek/OpenAI 的代码模板

```
import requests

def call_model(messages):
    url = "https://api.deepseek.com/chat/completions"
    headers = {
        "Authorization": "Bearer sk-你的key",
        "Content-Type": "application/json"
    }
    data = {
        "model": "deepseek-chat",
        "messages": messages,
        "temperature": 0.7,          # 随机性 (0-2)
        "max_tokens": 1024          # 最大回复长度
    }
    resp = requests.post(url, headers=headers, json=data, timeout=30)
    result = resp.json()
    return result["choices"][0]["message"]["content"]
```

关键参数

参数	作用
temperature	0=稳定确定，1=有创造力，2=放飞
max_tokens	模型最多输出多少字
system 消息	设定模型的角色和行为规范

第7课：完整的 RAG API（综合项目）

项目结构

```
rag_api.py
config.py
```

```
# 主服务: FastAPI + 知识库 + 调模型
# 配置文件: API Key
```

一次请求的完整链路

```
用户 POST /ask {"question": "加班有钱吗"}
  → search_docs() 检索知识库 → 找到相关文档
  → 拼接 prompt (资料 + 问题)
  → call_deepseek() 调真实模型
  → 返回 {"answer": "...", "source_docs": [...]}
```

学到了什么

1. 用 FastAPI 写一个 POST 接口
2. 用关键词/向量搜索检索本地知识
3. 把搜索结果拼成 prompt
4. 调真实大模型 API
5. 返回结构化结果

项目作品：公司知识库 RAG 问答系统

项目简介

一个基于 RAG 技术的企业知识库问答 API 服务。用户提问后，系统从知识库中检索相关文档，再调用大模型基于资料回答。

技术栈

- 后端框架：FastAPI (Python)
- 大模型：DeepSeek API
- 检索方式：关键词相似度搜索（可升级为向量数据库）
- 部署：uvicorn 本地启动

核心功能

- ✔ POST 接口接收问题
- ✔ 本地知识库检索
- ✔ 基于检索结果调用大模型
- ✔ 返回结构化 JSON（问题 + 答案 + 参考来源）
- ✔ Swagger 交互式 API 文档

可在简历中描述的亮点

"独立开发了基于 RAG 技术的企业知识库问答 API 服务。系统采用 FastAPI 搭建后端，对接 DeepSeek 大模型接口，实现了"检索→增强→生成"的完整链路。用户提问后可自动从知识库中检索相关文档，再基于资料生成准确回答，有效降低大模型幻觉。"

学习路线图进度

阶段	内容	状态
1. Python 基础语法	变量、列表、字典、JSON、函数	✔ 完成
2. 后端接口开发	FastAPI、POST/GET、请求响应	✔ 完成
3. 大模型 API 调用	DeepSeek 接口对接	✔ 完成
4. 提示词工程	角色设定、少样本、固定格式	✔ 完成
5. RAG 知识库	检索→增强→生成完整流程	✔ 完成
6. 向量数据库	待接入 Chroma/FAISS	📅 下一步
7. Agent 工具调用	让模型主动调工具	📅 下一步
8. Docker 部署	容器化上线	📅 后续
9. 流式输出	文字逐字输出效果	📅 后续

下一步学习建议

- 把关键词搜索换成真正的向量数据库（Chroma），这是企业级 RAG 的关键升级

2. **学 Agent**：让模型能主动调用工具（查天气、查数据库、发邮件）
3. **学流式输出**：实现像 ChatGPT 一样逐字回复的效果
4. **用 Docker 把项目部署上线**，变成真正可用的服务